Architecture 1001: x86-64 Assembly ForLoopNoRet.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

ForLoopNoRet.c - simple for loop

```c
#include <stdio.h>

int main(){
   int i;
   for(i = 0; i < 10; i++){
      printf("i = %d\n", i);
   }
   i--;
}
```

```
main:
0000000140001000  sub  rsp,38h
0000000140001004  mov  dword ptr [rsp+20h],0
000000014000100C  jmp  0000000140001018
000000014000100E  mov  eax,dword ptr [rsp+20h]
0000000140001012  inc  eax
0000000140001014  mov  dword ptr [rsp+20h],eax
0000000140001018  cmp  dword ptr [rsp+20h],0Ah
000000014000101D  jge  0000000140001031
000000014000101F  mov  edx,dword ptr [rsp+20h]
0000000140001023  lea  rcx,[0000000140006000h]
000000014000102A  call 00000001400010B0
000000014000102F  jmp  000000014000100E
0000000140001031  mov  eax,dword ptr [rsp+20h]
0000000140001035  dec  eax
0000000140001037  mov  dword ptr [rsp+20h],eax
000000014000103B  xor  eax,eax
000000014000103D  add  rsp,38h
0000000140001041  ret
```

ForLoopNoRet.c - simple for loop

```c
#include <stdio.h>

int main(){
   int i;
   for(i = 0; i < 10; i++){
      printf("i = %d\n", i);
   }
   i--;
}
```

```
main:
0000000140001000  sub  rsp,38h
0000000140001004  mov  dword ptr [rsp+20h],0
000000014000100C  jmp  0000000140001018
000000014000100E  mov  eax,dword ptr [rsp+20h]
0000000140001012  inc  eax
0000000140001014  mov  dword ptr [rsp+20h],eax
0000000140001018  cmp  dword ptr [rsp+20h],0Ah
000000014000101D  jge  0000000140001031
000000014000101F  mov  edx,dword ptr [rsp+20h]
0000000140001023  lea  rcx,[0000000140006000h]
000000014000102A  call 00000001400010B0
000000014000102F  jmp  000000014000100E
0000000140001031  mov  eax,dword ptr [rsp+20h]
0000000140001035  dec  eax
0000000140001037  mov  dword ptr [rsp+20h],eax
000000014000103B  xor  eax,eax
000000014000103D  add  rsp,38h
0000000140001041  ret
```

ForLoopNoRet.c - simple for loop

```c
#include <stdio.h>

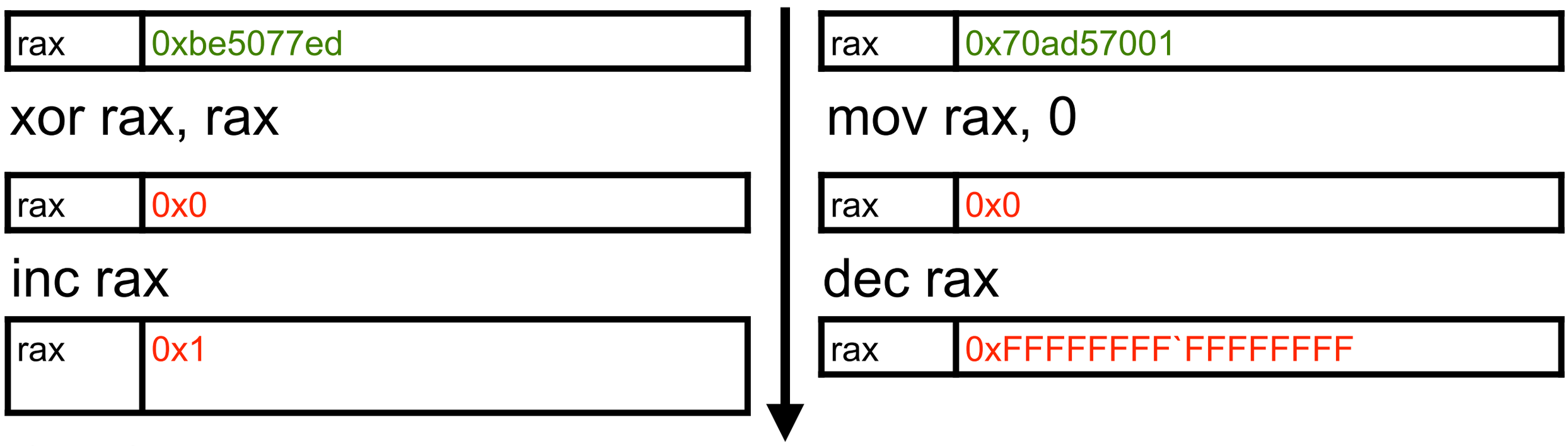
int main(){
   int i;
   for(i = 0; i < 10; i++){
      printf("i = %d\n", i);
   }
   i--;
}
```

```
main:
0000000140001000  sub  rsp,38h
0000000140001004  mov  dword ptr [rsp+20h],0
000000014000100C  jmp  0000000140001018
000000014000100E  mov  eax,dword ptr [rsp+20h]
0000000140001012  inc  eax
0000000140001014  mov  dword ptr [rsp+20h],eax
0000000140001018  cmp  dword ptr [rsp+20h],0Ah
000000014000101D  jge  0000000140001031
000000014000101F  mov  edx,dword ptr [rsp+20h]
0000000140001023  lea  rcx,[0000000140006000h]
000000014000102A  call 00000001400010B0
000000014000102F  jmp  000000014000100E
0000000140001031  mov  eax,dword ptr [rsp+20h]
0000000140001035  dec  eax
0000000140001037  mov  dword ptr [rsp+20h],eax
000000014000103B  xor  eax,eax
000000014000103D  add  rsp,38h
0000000140001041  ret
```



INC/DEC - Increment / decrement

- Single source/destination operand can be r/mX
- Increase or decrease the value by 1
- When optimized, compilers will tend to favor *not* using inc/dec, as directed by the Intel optimization guide. So their presence may be indicative of hand-written, or un-optimized code
- Modifies OF, SF, ZF, AF, PF, and CF flags



Book page 59



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

ForLoopNoRet.c Takeaways

- For loops will be some combination of a JCC (to determine if the exit condition is met yet), and an absolute JMP (to return from the end to the conditional checking code)
- In the absence of an explicit return value VS & GCC default to returning 0

```c
#include <stdio.h>

int main(){
  int i;
  for(i = 0; i < 10; i++){
    printf("i = %d\n", i);
  }
  i--;
}
```

```
main:
0000000140001000  sub  rsp,38h
0000000140001004  mov  dword ptr [rsp+20h],0
000000014000100C  jmp  0000000140001018
000000014000100E  mov  eax,dword ptr [rsp+20h]
0000000140001012  inc  eax
0000000140001014  mov  dword ptr [rsp+20h],eax
0000000140001018  cmp  dword ptr [rsp+20h],0Ah
000000014000101D  jge  0000000140001031
000000014000101F  mov  edx,dword ptr [rsp+20h]
0000000140001023  lea  rcx,[0000000140006000h]
000000014000102A  call 00000001400010B0
000000014000102F  jmp  000000014000100E
0000000140001031  mov  eax,dword ptr [rsp+20h]
0000000140001035  dec  eax
0000000140001037  mov  dword ptr [rsp+20h],eax
000000014000103B  xor  eax,eax
000000014000103D  add  rsp,38h
0000000140001041  ret
```

Instructions we now know (21)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP
- AND/OR/XOR/NOT
- **INC/DEC**

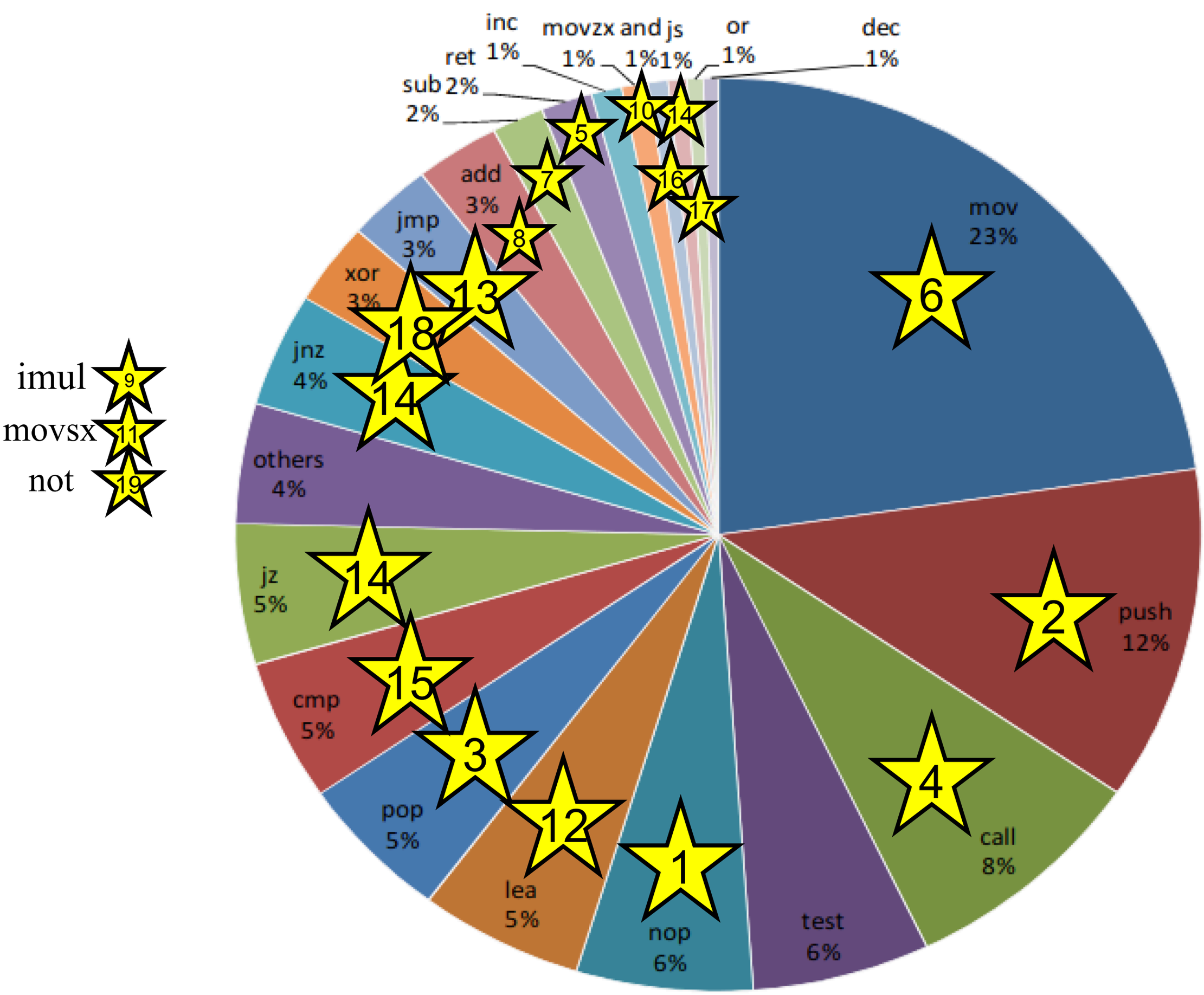


Fig. 2 Web browsers' instruction frequencies

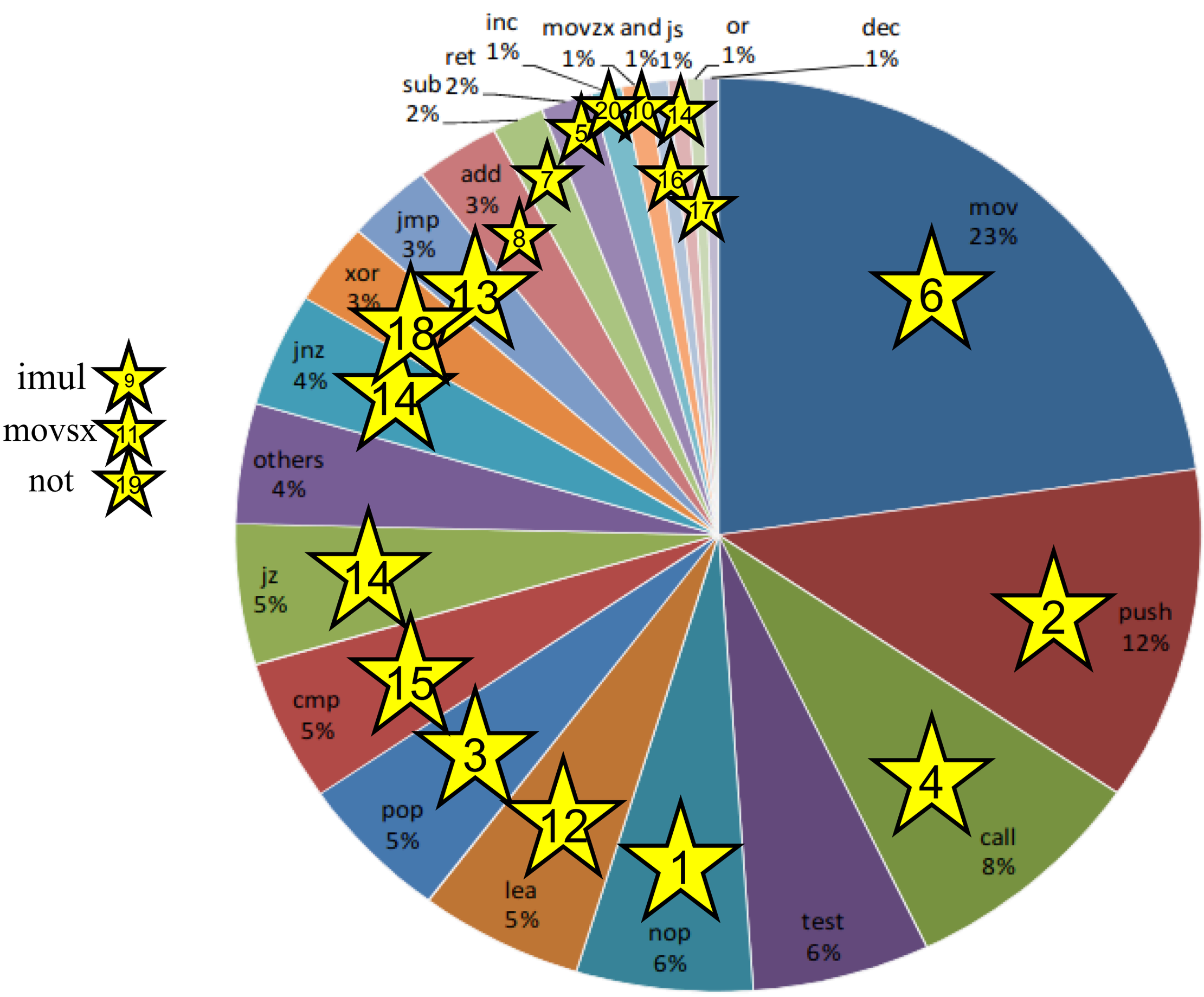


Fig. 2 Web browsers' instruction frequencies

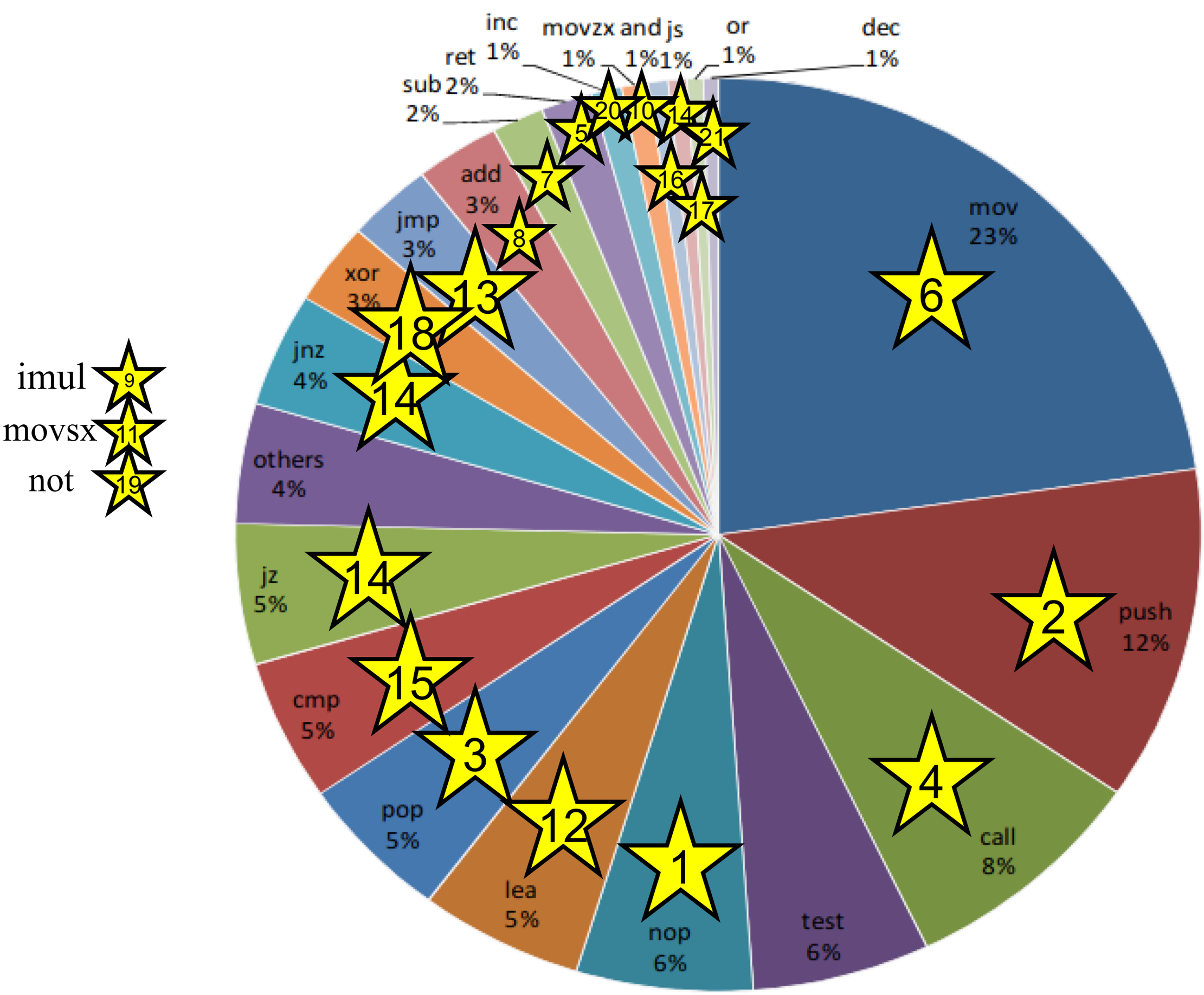


Fig. 2 Web browsers' instruction frequencies